

作业 HW5* 实验报告

姓名：汪嘉晨 学号：2353900 日期：2024 年 12 月 16 日

1. 涉及数据结构和相关背景

“查找”是计算机科学中一种重要的操作，指的是在数据集中寻找某个特定元素或满足某种条件的元素。它通常与数据结构密切相关，数据结构的选择和实现方式会影响查找操作的效率。我们可以从以下几个方面来探讨“查找”以及相关的背景：

1. 查找的基本概念

查找操作通常分为两类：

- **顺序查找**：从数据集的第一个元素开始，逐个比较每个元素，直到找到目标元素为止。时间复杂度为 $O(n)$ ，其中 n 是数据集中元素的数量。
- **二分查找**：要求数据集必须是有序的，通过反复将查找范围缩小一半来查找目标元素。时间复杂度为 $O(\log n)$ ，比顺序查找要高效得多。

2. 查找与数据结构的联系

查找操作与所选择的数据结构密切相关，不同的数据结构适用于不同类型的查找任务。常见的数据结构有：

- **数组 (Array)**：数组的顺序查找非常简单，但如果数组有序，二分查找能提供更高效率的查找方式。数组的查找时间复杂度分别是 $O(n)$ (顺序查找) 和 $O(\log n)$ (二分查找)。
- **链表 (Linked List)**：链表进行顺序查找，时间复杂度也是 $O(n)$ ，没有直接的方式进行快速查找，因为没有索引结构。
- **二叉搜索树 (Binary Search Tree, BST)**：一种带有排序规则的树形结构，查找操作可以通过比较当前节点与目标值来决定是否进入左子树或右子树。平均时间复杂度是 $O(\log n)$ ，但在树不平衡时，最坏情况下为 $O(n)$ 。
- **哈希表 (Hash Table)**：哈希表通过哈希函数将元素映射到一个位置，从而使得查找操作平均时间复杂度为 $O(1)$ ，非常高效。但哈希表也可能出现哈希冲突，需要通过链表或开放地址法来处理。
- **平衡树 (如 AVL 树、红黑树等)**：这些是自平衡的二叉搜索树，保证在插入和删除操作后，树的高度始终保持在对数级别，查找操作的时间复杂度始终为 $O(\log n)$ 。

3. 查找的应用背景

查找不仅仅是理论上的概念，它在实际应用中有广泛的应用场景：

- **数据库查询**：数据库中数据存储常常采用 B 树或哈希索引来提高查找效率。查询操作是数据库的核心之一。
- **搜索引擎**：搜索引擎中的数据索引也利用了查找操作来快速定位相关的网页或文档。
- **缓存系统**：许多缓存系统（如 Redis）会通过哈希表或树形结构来快速查找数据，避免每次访问时都从磁盘读取。

4. 查找优化

查找算法的效率在不同的情况下可能会有很大的差异，因此在实际应用中，常常需要根据数据的特性来选择最适合的查找策略。例如：

- 如果数据集是静态且有序的，可以使用**二分查找**。
- 如果数据集是动态变化的且要求高效插入和查找，可以选择使用**哈希表**或**平衡树**。
- 在一些非常大的数据集上，可能会使用更复杂的查找结构，如**B+树**，特别是在文

件系统和数据库中。

5. 查找相关的算法

- **线性查找（顺序查找）**：最简单的查找方式，逐个比对元素。
- **二分查找（Binary Search）**：在有序数据中使用的高效查找方式，通过将数据集一分为二来不断缩小查找范围。
- **哈希查找**：使用哈希函数将数据映射到哈希表的槽中，能够提供常数时间复杂度的查找操作。
- **跳表（Skip List）**：跳表是一种基于链表的概率型数据结构，通过多级索引来加速查找操作，通常用于替代平衡树。

总结来说，查找在计算机科学中是一个非常基础且重要的操作，其效率直接影响到程序的整体性能。选择合适的数据结构和查找算法是进行性能优化时的关键因素。

2. 实验内容

2.1 和有限的最长子序列

2.1.1 问题描述

给你一个长度为 n 的整数数组 `nums` 和一个长度为 m 的整数数组 `queries`，返回一个长度为 m 的数组 `answer`，其中 `answer[i]` 是 `nums` 中元素之和小于等于 `queries[i]` 的子序列的最大长度。子序列是由一个数组删除某些元素（也可以不删除）但不改变元素顺序得到的一个数组。

2.1.2 基本要求

输入

第一行包括两个整数 n 和 m ，分别表示数组 `nums` 和 `queries` 的长度

第二行包括 n 个整数，为数组 `nums` 中元素

第三行包含 m 个整数，为数组 `queries` 中元素

对于 20% 的数据，有 $1 \leq n, m \leq 10$

对于 40% 的数据，有 $1 \leq n, m \leq 100$

对于 100% 的数据，有 $1 \leq n, m \leq 1000$

对于所有数据， $1 \leq \text{nums}[i], \text{queries}[i] \leq 106$

下载编译并运行 `p126_data.cpp` 以生成随机测试数据

输出

输出一行，包括 m 个整数，为 `answer` 中元素

2.1.3 数据结构设计（数组）

```
int n, m;
// 读取数组 nums 和 queries 的长度
cin >> n >> m;
vector<int> nums(n);
vector<int> queries(m);
```

2.1.4 功能说明（函数、类）

```
// 对 nums 进行排序，以便于找到满足条件的最长子序列
```

```

sort(nums.begin(), nums.end());
vector<int> answer(m);
// 对每个查询进行处理
for (int i = 0; i < m; ++i) {
    int sum = 0;
    int count = 0;
    // 计算nums 中元素之和小于等于 queries[i] 的子序列的最大长度
    for (int j = 0; j < n; ++j) {
        if (sum + nums[j] <= queries[i]) {
            sum += nums[j];
            count++;
        }
        else {
            break;
        }
    }
    answer[i] = count;
}

```

2.1.5 调试分析（遇到的问题 and 解决方法）

问题：排序？

解决方法：直接使用现成代码

```
sort(nums.begin(), nums.end());
```

快速排序（QuickSort）是一种高效的排序算法，采用分治法（Divide and Conquer）策略。其基本思想是通过一趟排序将要排序的数据分割成独立的两部分，其中一部分的所有数据都比另一部分的所有数据都要小，然后再按此方法对这两部分数据分别进行快速排序，整个排序过程可以递归进行，以此达到整个数据变成有序序列。

2.1.6 总结和体会

排序+前缀和贪心算法

2.2 二叉排序树

2.2.1 问题描述

描述

二叉排序树 BST（二叉查找树）是一种动态查找表，基本操作集包括：创建、查找，插入，删除，查找最大值，查找最小值等。

本题实现一个维护整数集合（允许有重复关键字）的 BST，并具有以下功能：1. 插入一个整数 2. 删除一个整数 3. 查询某个整数有多少个 4. 查询最小值 5. 查询某个数字的前驱（集合中比该数字小的最大值）。

2.2.2 基本要求

输入

第 1 行一个整数 n，表示操作的个数；

接下来 n 行，每行一个操作，第一个数字 op 表示操作种类：

- 若 op=1，后面跟着一个整数 x，表示插入数字 x
- 若 op=2，后面跟着一个整数 x，表示删除数字 x（若存在则删除，否则输出 None，若有多个则只删除一个），
- 若 op=3，后面跟着一个整数 x，输出数字 x 在集合中有多少个（若 x 不在集合中则输出 0）
- 若 op=4，输出集合中的最小值（保证集合非空）
- 若 op=5，后面跟着一个整数 x，输出 x 的前驱（若不存在前驱则输出 None，x 不一定在集合中）

输出

一个操作输出 1 行（除了插入操作没有输出）

2.2.3 数据结构设计

```
// 定义二叉排序树的节点结构
struct Node {
    int key;           // 节点的键值
    int count;         // 相同键值的数量
    Node* left;        // 左子节点指针
    Node* right;        // 右子节点指针
    Node(int k) : key(k), count(1), left(nullptr), right(nullptr) {}
};
```

2.2.4 功能说明（函数、类）

```
// 插入节点
Node* insert(Node* root, int key) {
    if (root == nullptr) {
        // 如果根节点为空，创建新节点
        return new Node(key);
    }
    if (key == root->key) {
        // 键值相同，计数加一
        root->count++;
    }
    else if (key < root->key) {
        // 键值小于当前节点，递归插入左子树
        root->left = insert(root->left, key);
    }
    else {
        // 键值大于当前节点，递归插入右子树
        root->right = insert(root->right, key);
    }
    return root;
}

// 查找最小值节点
```

```

Node* findMin(Node* root) {
    while (root->left != nullptr) {
        root = root->left;
    }
    return root;
}

// 删除节点
Node* remove(Node* root, int key, bool& found) {
    if (root == nullptr) {
        // 未找到要删除的节点
        found = false;
        return nullptr;
    }
    if (key < root->key) {
        // 在左子树中删除
        root->left = remove(root->left, key, found);
    }
    else if (key > root->key) {
        // 在右子树中删除
        root->right = remove(root->right, key, found);
    }
    else {
        // 找到要删除的节点
        found = true;
        if (root->count > 1) {
            // 如果节点有重复，计数减一
            root->count--;
        }
        else {
            if (root->left != nullptr && root->right != nullptr) {
                // 节点有两个子节点，找到右子树的最小值替代
                Node* minNode = findMin(root->right);
                root->key = minNode->key;
                root->count = minNode->count;
                bool temp;
                // 删除右子树中的最小值节点
                root->right = remove(root->right, minNode->key, temp);
            }
            else {
                // 节点有一个或零个子节点
                Node* temp = root;
                if (root->left != nullptr) {
                    // 用左子节点替代
                    root = root->left;
                }
            }
        }
    }
}

```

```

        }
        else {
            // 用右子节点替代
            root = root->right;
        }
        delete temp; // 释放内存
    }
}

return root;
}

// 查询键值的数量
int queryCount(Node* root, int key) {
    if (root == nullptr) {
        // 未找到, 返回0
        return 0;
    }
    if (key == root->key) {
        // 找到, 返回计数
        return root->count;
    }
    else if (key < root->key) {
        // 在左子树中查询
        return queryCount(root->left, key);
    }
    else {
        // 在右子树中查询
        return queryCount(root->right, key);
    }
}

// 查找最小值
int findMinValue(Node* root) {
    Node* minNode = findMin(root);
    return minNode->key;
}

// 查找前驱节点
int findPredecessor(Node* root, int key, int pred = -1) {
    if (root == nullptr) {
        // 未找到前驱, 返回当前前驱值
        return pred;
    }
    if (key == root->key) {
        if (root->left != nullptr) {
            // 前驱为左子树的最右节点

```

```

        Node* temp = root->left;
        while (temp->right != nullptr) {
            temp = temp->right;
        }
        return temp->key;
    }
    else {
        // 无左子树，前驱为已记录的 pred
        return pred;
    }
}
else if (key < root->key) {
    // 在左子树中查找
    return findPredecessor(root->left, key, pred);
}
else {
    // 更新前驱为当前节点，继续在右子树中查找
    pred = root->key;
    return findPredecessor(root->right, key, pred);
}
}
}

```

2.2.5 调试分析（遇到的问题 and 解决方法）

问题：二叉排序树变成允许增加重复数字的二叉树

解决方法：只需要在结构体里面加入 `int count;` 即可

2.2.6 总结和体会

1. 数据结构选择：

使用二叉排序树（BST）来维护整数集合，能够高效地进行插入、删除和查找操作。

每个节点包含键值、计数器（用于处理重复元素）以及左右子节点指针。

2. 操作实现：

插入操作：递归地将新值插入到合适的位置，如果值已存在则增加计数器。

删除操作：递归地找到要删除的节点，处理三种情况：节点有两个子节点、一个子节点或没有子节点。

查询数量：递归地查找指定值的节点，返回计数器值。

查询最小值：通过不断访问左子节点找到最小值节点。

查询前驱：递归地查找前驱节点，前驱是左子树的最右节点或已记录的前驱值。

3. 边界条件处理：

插入和删除操作需要处理空树的情况。

删除操作需要处理节点有重复值的情况。

查询操作需要处理节点不存在的情况。

2.3 换座位

2.3.1 问题描述

期末考试，监考老师粗心拿错了座位表，学生已经落座，现在需要按正确的座位表给学生重新排座。假设一次交换你可以选择两个学生并让他们交换位置，给你原来错误的座位表和正确的座位表，问给学生重新排座需要最少的交换次数。

2.3.2 基本要求

输入描述

两个 $n \times m$ 的字符串数组，表示错误和正确的座位表 `old_chart` 和 `new_chart`，`old_chart[i][j]` 为原来坐在第 i 行第 j 列的学生名字

对于 100% 的数据， $1 \leq n, m \leq 200$;

人名为仅由小写英文字母组成的字符串，长度不大于 5

下载编译并运行 `p138_data.cpp` 生成随机测试数据

输出描述

一个整数，表示最少交换次数

2.3.3&4 数据结构设计

```
class Solution {
public:
    int solve(std::vector<std::vector<std::string>>& old_chart, std::vector<std::vector<std::string>>& new_chart) {
        int n = old_chart.size();
        int m = old_chart[0].size();
        std::map<std::string, std::pair<int, int>> old_positions;
        std::map<std::string, std::pair<int, int>> new_positions;
        // 记录每个学生在 old_chart 中的位置
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                old_positions[old_chart[i][j]] = { i, j };
            }
        }
        // 记录每个学生在 new_chart 中的位置
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                new_positions[new_chart[i][j]] = { i, j };
            }
        }
        std::vector<bool> visited(n * m, false);
        int swaps = 0;
        // 遍历每个学生，找到所有的环
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                if (!visited[i * m + j]) {
                    int cycle_length = 0;
                    int x = i, y = j;
                    // 找到一个环
```



```

        while (!visited[x * m + y]) {
            visited[x * m + y] = true;
            auto new_pos = new_positions[old_chart[x][y]];
            x = new_pos.first;
            y = new_pos.second;
            ++cycle_length;
        }
        // 环的长度减一就是所需的交换次数
        if (cycle_length > 1) {
            swaps += cycle_length - 1;
        }
    }
}
return swaps;
};

```

2.3.5 调试分析（遇到的问题和解决方法）

使用环的概念是因为学生的座位交换可以被看作是一个置换（permutation）。置换可以分解成若干个不相交的环，每个环表示一组学生的位置互相交换，最终回到起点。

具体来说，假设我们有一个学生从位置 A 移动到位置 B，位置 B 的学生移动到位置 C，位置 C 的学生又移动到位置 A，这样就形成了一个环。通过找到所有这样的环，我们可以计算出最少的交换次数。

1. 置换的性质：

任何置换都可以分解成若干个不相交的环。例如，置换(1 2 3)(4 5)表示 1 到 2，2 到 3，3 到 1，4 到 5，5 到 4。

2. 环的长度与交换次数的关系：

一个长度为 k 的环需要 k-1 次交换才能完成。例如，长度为 3 的环需要 2 次交换。

3. 遍历所有位置：

通过遍历所有位置并找到所有的环，我们可以确保每个学生都被正确地重新安排到新座位。

2.3.6 总结和体会

1. 记录位置：

首先，程序使用两个 std::map 来记录每个学生在旧座位表和新座位表中的位置。old_positions 记录每个学生在 old_chart 中的位置，new_positions 记录每个学生在 new_chart 中的位置。

2. 标记访问过的位置：

使用一个 std::vector<bool>来标记每个位置是否已经访问过。这个向量的大小是 n * m，其中 n 是行数，m 是列数。

3. 寻找环：

程序遍历每个位置，寻找所有的环。一个环表示一组学生的位置互相交换，最终回到起点。

对于每个未访问过的位置，程序会沿着学生的新位置不断移动，直到回到起点，形成一个环。

4. 计算交换次数：

对于每个环，环的长度减一就是所需的交换次数。例如，一个长度为 3 的环需要 2 次交换才能完

成。

2.6 最大频率栈

2.6.1 问题描述

设计一个类似堆栈的数据结构，将元素推入堆栈，并从堆栈中弹出出现频率最高的元素。

实现 FreqStack 类：

FreqStack() 构造一个空的堆栈。

void push(int val) 将一个整数 val 压入栈顶。

int pop() 删除并返回堆栈中出现频率最高的元素。如果出现频率最高的元素不只一个，则移除并返回最接近栈顶的元素。

2.6.2 基本要求

输入描述

第一行包含一个整数 n

接下来 n 行每行包含一个字符串（push 或 pop）表示一个操作，若操作为 push，则该行额外包含一个整数 val，表示压入堆栈的元素

对于 100% 的测试数据， $1 \leq n \leq 20000$ ， $0 \leq val \leq 10^9$ ，且当堆栈为空时不会输入 pop 操作

输出描述

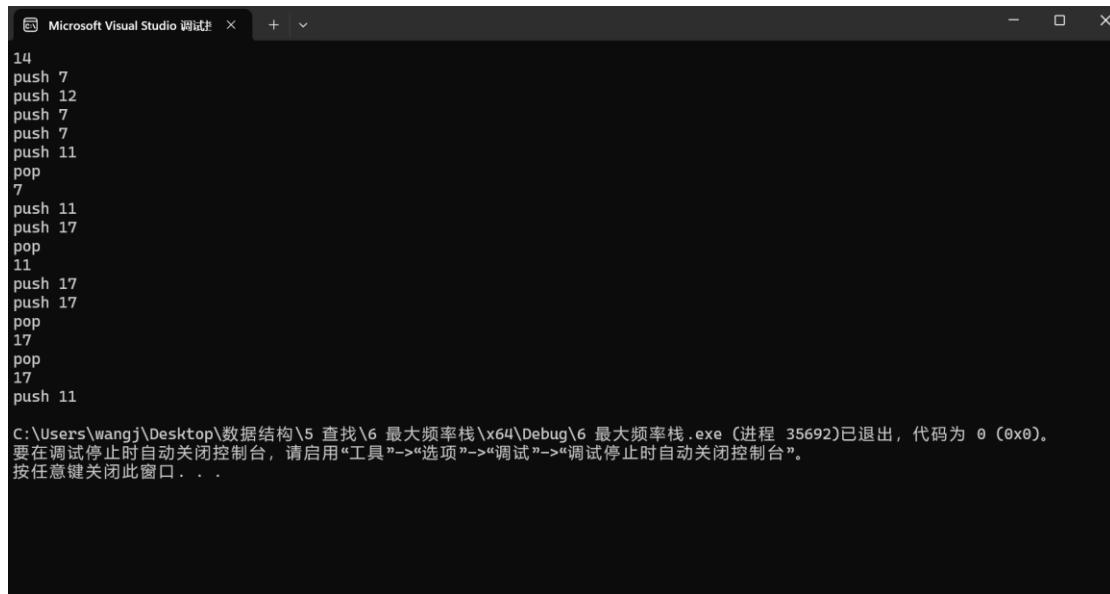
输出包含若干行，每有一个 pop 操作对应一行，为弹出堆栈的元素

2.6.3&4 数据结构设计

```
class FreqStack {
private:
    map<int, int> freq; // 记录每个值的频率
    map<int, stack<int>> group; // 记录每个频率对应的值的栈
    int maxFreq; // 当前的最大频率
public:
    FreqStack() : maxFreq(0) {}
    void push(int val) {
        int f = ++freq[val];
        if (f > maxFreq) {
            maxFreq = f;
        }
        group[f].push(val);
    }
    int pop() {
        int val = group[maxFreq].top();
        group[maxFreq].pop();
        if (group[maxFreq].empty()) {
            maxFreq--;
        }
        freq[val]--;
        return val;
    }
};
```

```
}  
};
```

2.6.5 调试分析（遇到的问题和解决方法）



```
14  
push 7  
push 12  
push 7  
push 7  
push 11  
pop  
7  
push 11  
push 17  
pop  
11  
push 17  
push 17  
pop  
17  
pop  
17  
push 11
```

C:\Users\wangj\Desktop\数据结构\5 查找\6 最大频率栈\x64\Debug\6 最大频率栈.exe (进程 35692)已退出，代码为 0 (0x0)。
要在调试停止时自动关闭控制台，请启用“工具”->“选项”->“调试”->“调试停止时自动关闭控制台”。
按任意键关闭此窗口...

2.6.6 总结和体会

1. **频率记录 (freq)**：使用一个哈希表 (`map<int,int>`) 记录每个元素的出现频率，即元素 `val` 出现了多少次。
2. **分组栈 (group)**：将具有相同出现频率的元素放入同一个栈中，使用哈希表 (`map<int, stack<int>>`) 以频率为键，栈为值。例如，`group[2]` 存放所有出现频率为 2 的元素。
3. **最大频率 (maxFreq)**：记录当前元素的最大出现频率。

push 操作流程：

- 增加元素 `val` 的出现频率 `freq[val]`。
- 更新最大频率 `maxFreq` (如果 `freq[val]` 超过了当前 `maxFreq`)。
- 将元素 `val` 压入对应频率的栈 `group[freq[val]]`。

pop 操作流程：

- 从最大频率对应的栈 `group[maxFreq]` 中弹出元素 `val`。
- 减少元素 `val` 的出现频率 `freq[val]`。
- 如果 `group[maxFreq]` 为空，减少 `maxFreq` 的值。

通过这种方式，栈始终能够在 $O(1)$ 的时间复杂度内完成 `push` 和 `pop` 操作，且保证 `pop` 操作总是返回频率最高且最近压入的元素。

3.实验总结

在学习数据结构和算法的过程中，查找操作是最基础也是最重要的内容之一。从顺序查找到二分查找，再到更复杂的哈希表和树形结构，每种查找方法和数据结构背后都有着不同的设计思想和应用场景。查找算法并非孤立存在，它与排序、动态规划、图算法等有着密切的关系。例如，在某些情况下，排序是提高查找效率的前提条件；而在图的遍历中，查找操作也是遍历过程中不可或缺的一部分。